ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

КУРСОВОЙ ПРОЕКТ

ПМ.02 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

МДК.02.01. Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

РАЗРАБОТКА И СОПРОВОЖДЕНИЕ ВИРТУАЛЬНОЙ СИСТЕМЫ СБОРА И УЧЁТА ДАННЫХ НА ARDUINO «ВАРИАНТ №7»

Выполнила:

Егорова Александра Александровна

студентка:	54-15 группы
Специальность:	09.02.08 Интеллектуальные интегрированные системы
Руководители:	*Борисов Егор Николаевич* *Дадаев Матвей Васильевич*
Оценка	

Санкт-Петербург, 2026

ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ
Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

УТВЕРЖДАЮ

Заведующий отделением информационных технологий А.Ю. Бычков

_____

«_____» _____ 2026 г.

З А Д А Н И Е

для выполнения курсового проекта

Специальность:	09.02.08 «Интеллектуальные интегрированные системы»
Группа:	54-15
Ф.И.О. студента	**Егорова Александра Александровна**

Тема курсового проекта:

Разработка и сопровождение виртуальной системы сбора и учёта данных на Arduino «Вариант №7»

Разработка принципиальной схемы в EasyEDA, схемы на Arduino Uno в симуляторе Wokwi, прошивки, базы данных SQLite, загрузки и контроля данных в Node-RED, ведение проекта в Git. Входные данные варианта (заполняются по таблице вариантов, Приложение А):

Сценарий	Платёжный терминал
Датчик 1 (аналоговый)	NTC (корпус, 0–80 °C)
Датчик 2	US (расстояние до клиента)
Датчик 3	SW (крышка корпуса)
Исполнительное устройство	реле (вентилятор)
Светодиод-индикатор (цвет, ток)	жёлтый, 10 мА
Период опроса T	3 с
Порог события	температура > 40 °C
Дата выдачи задания для выполнения курсового проекта	«13» октября 2026 г.
Срок защиты курсового проекта	«5» ноября 2026 г.

Содержание курсового проекта

Введение
1. Постановка задачи и структурная схема
2. Принципиальная схема в EasyEDA и схема в Wokwi
3. Расчётная часть
    3.1. Расчёт резистора светодиода
    3.2. Расчёт шага АЦП
    3.3. Расчёт объёма данных
4. Программа микроконтроллера
5. База данных
6. Обработка данных в Node-RED
7. Тестирование и проверка неисправности
8. Предложения по модернизации

Выводы
Список используемых сокращений Список используемой литературы
ПРИЛОЖЕНИЯ
    А. Структурная схема
    Б. Принципиальная схема (EasyEDA) В. Схема в Wokwi (скриншот)
    Г. Листинг скетча
    Д. Схема базы данных (ER-диаграмма и schema.sql) Е. Flow Node-RED (flow.json)

Список рекомендованной литературы и источников

1. Чакон С., Штрауб Б. Pro Git. Электронная книга: https://git-scm.com/book/ru/v2
2. Документация Wokwi: https://docs.wokwi.com
3. Справочник языка Arduino: https://www.arduino.cc/reference/
4. Документация Node-RED: https://nodered.org/docs/
5. Документация SQLite: https://www.sqlite.org/docs.html

Руководитель курсового проекта _____ (_____)
Подпись Расшифровка подписи

Студент _____ (_____)
Подпись Расшифровка подписи

Содержание

Введение 6
1. Постановка задачи 8
2. Структура и алгоритм работы системы 10
2.1. Структурная схема 10
2.2. Алгоритм функционирования 11
2.3. Протокол обмена данными 12
3. Разработка аппаратной части 13
3.1. Выбор компонентов и подключение 13
3.2. Расчёт резисторов и оценка потребления 14
3.3. Принципиальная схема в EasyEDA 14
3.4. Модель в Wokwi 15
4. Разработка прошивки 16
5. Проектирование базы данных 17
5.1. Модель данных 17
5.2. Описание таблиц 17
5.3. Контрольные запросы 18
6. Обработка данных в Node-RED 19
6.1. Состав потока 19
6.2. Функция разбора журнала 20
6.3. Запуск 20
7. Тестирование и проверка неисправностей 21
7.1. Штатный сценарий 21
7.2. Сценарий неисправности датчика 23
7.3. Сводка результатов 23
8. Расчёт объёма данных 24
9. Управление версиями Git 25
10. Предложения по модернизации 26
Выводы 26
Список сокращений и обозначений 30
Список используемой литературы 31
Приложение А Структурная схема 33
Приложение Б Схема электрическая принципиальная 34
Приложение В Перечень элементов 35
Приложение Г Модель в Wokwi 36
Приложение Д Листинг программы Arduino (variant_7.ino) 37
Приложение Е Листинг функции разбора журнала Node-RED 39
Приложение Ж Скрипты базы данных 41
Приложение И Журнал Serial и загрузочный скрипт data/log.txt (штатный сценарий) 42

Приложение К История Git..... 43

Введение

Интеллектуальные интегрированные системы (ИИС) находят широкое применение в сфере самообслуживания, обеспечивая автоматизацию процессов и контроль технического состояния оборудования. Типовой задачей сопровождения таких систем является мониторинг критических параметров среды и состояния узлов устройства для предотвращения поломок и обеспечения безопасности пользователей. Платёжные терминалы, работающие в круглосуточном режиме, требуют особого внимания к температурному режиму, так как перегрев электронных компонентов может привести к сбоям в обработке транзакций или выходу оборудования из строя.

Прежде чем изготавливать физический прототип, целесообразно спроектировать и отладить систему в виртуальной среде. Принципиальная схема разрабатывается в системе EasyEDA, алгоритмы работы микроконтроллера проверяются в симуляторе Wokwi, данные измерений сохраняются в базе данных SQLite, а автоматизация загрузки и анализа информации возлагается на среду Node-RED. Весь процесс разработки сопровождается системой контроля версий Git. Такой подход позволяет полностью воспроизвести проект без использования дорогостоящего оборудования на этапе проектирования.

Объектом данного курсового проекта является виртуальная система сбора и учёта данных на базе платы Arduino Uno для сценария «Платёжный терминал». Предметом исследования выступают схемотехнические решения, программное обеспечение микроконтроллера, структура базы данных и средства автоматизированной обработки данных, обеспечивающие контроль температуры корпуса, расстояния до клиента и состояния крышки терминала.

Целью проекта является разработка и документирование работоспособной виртуальной системы, которая осуществляет мониторинг

температуры внутри корпуса терминала, фиксирует присутствие пользователя и контролирует целостность конструкции, а также автоматически сохраняет показания в базу данных и сигнализирует о перегреве.

Для достижения поставленной цели необходимо решить следующие задачи:

- разработать структурную схему и алгоритм функционирования системы;

- выбрать компоненты, рассчитать токоограничивающие резисторы и выполнить принципиальную схему в EasyEDA в соответствии с требованиями ЕСКД;

- собрать модель в симуляторе Wokwi и написать программу для Arduino Uno;

- спроектировать базу данных SQLite и набор контрольных запросов для анализа статистики;

- настроить поток Node-RED для автоматического разбора журнала, загрузки данных в базу и фиксации событий перегрева;

- провести тестирование системы на штатном и аварийном сценариях;

- оформить проект в репозитории Git и подготовить пояснительную записку.

1. Постановка задачи

Разрабатывается система мониторинга платёжного терминала на Arduino Uno. NTC измеряет температуру корпуса (0–80 °C), HC-SR04 — расстояние до клиента (2–400 см), концевой выключатель контролирует крышку (0/1). Реле управляет вентилятором, жёлтый светодиод сигнализирует о превышении температуры 40 °C. Основные параметры приведены в таблице 1.

Таблица 1 – Входные данные варианта №7

Параметр	Значение
Сценарий	Платёжный терминал
Датчик 1 (аналоговый), s1	NTC – температура корпуса, диапазон 0–80 ℃
Датчик 2, s2	US (HC-SR04) – расстояние до клиента, диапазон 2–400 см
Датчик 3, s3	SW (концевой выключатель) – крышка корпуса, значения 0 / 1
Исполнительное устройство	Реле K1, управляющее вентилятором
Светодиод-индикатор	Жёлтый, рабочий ток 10 мА
Период опроса T	3 с
Порог события	Температура s1 > 40 ℃

Система должна выполнять следующие функции:

1. каждые T = 3 с опрашивать три датчика и передавать результат в последовательный порт строкой формата «DATA»;
2. при повышении температуры выше 40 °C включать реле вентилятора и жёлтый светодиод и передавать строку «EVENT» с типом THRESHOLD; при снижении температуры до нормы выключать их и передавать EVENT с типом NORMAL;

3. сохранять строки последовательного порта в текстовый журнал data/log.txt;

4. автоматически загружать журнал в базу SQLite (таблицы readings и events);

5. при значении датчика вне допустимого диапазона записывать в таблицу событий неисправность FAULT;

6. выдавать три контрольных запроса: десять последних показаний; минимум, среднее и максимум по каждому датчику; список всех событий.

Результатами работы являются: принципиальная схема в EasyEDA, модель в Wokwi, программа Arduino, база данных, поток Node-RED, репозиторий Git и настоящая пояснительная записка с приложениями.

2. Структура и алгоритм работы системы

2.1. Структурная схема

Система состоит из трёх датчиков, микроконтроллера Arduino Uno, исполнительного устройства и программной цепочки обработки данных. Датчики подключены к входам платы; плата управляет реле вентилятора и светодиодом-индикатором и передаёт по последовательному порту (UART) текстовые строки. Строки сохраняются в журнале data/log.txt, откуда Node-RED загружает их в базу данных SQLite.



Рисунок 1 – Структурная схема системы

Структурная схема в увеличенном виде приведена в приложении А.

2.2. Алгоритм функционирования

После инициализации UART и выводов контроллер циклически считывает датчики, увеличивает n и выводит DATA. При s1 > 40 °C и неактивном alarm включаются реле и светодиод, передаётся THRESHOLD. При s1 =< 40 °C и активном alarm устройства выключаются и передаётся NORMAL. События формируются только при изменении состояния. Интервал опроса T = 3 с реализован через millis().

Логика смены состояния исключает повторную отправку одинаковых событий в соседних циклах

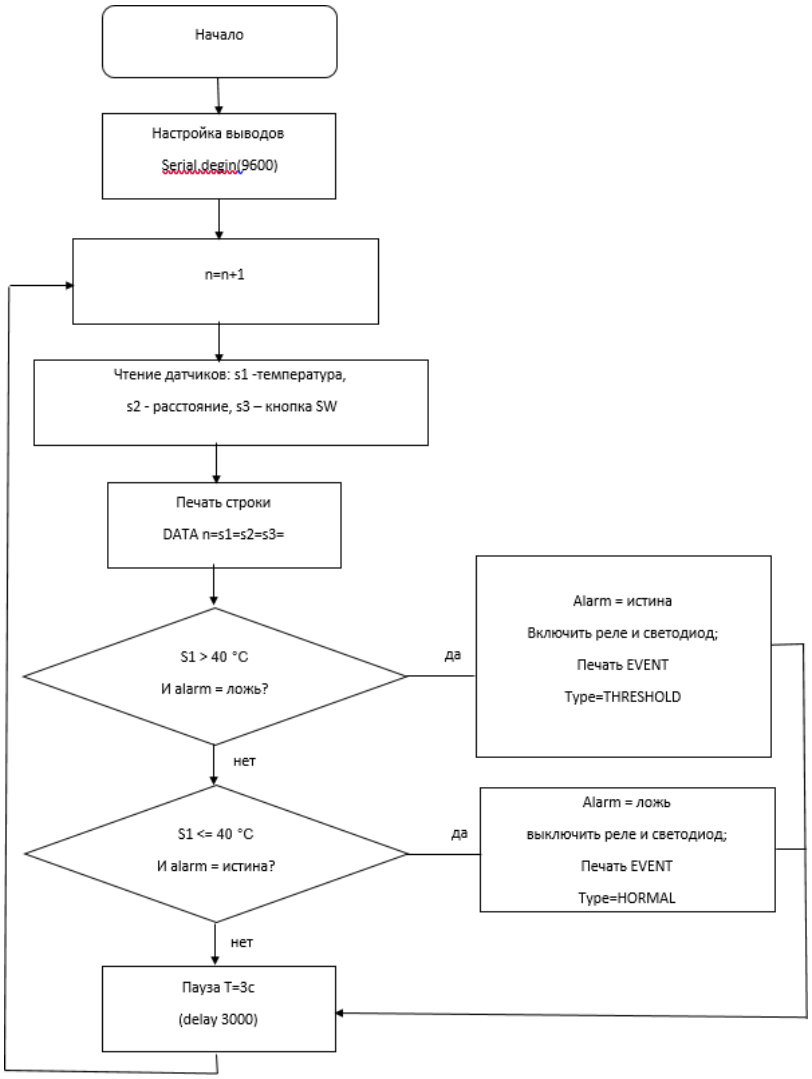


Рисунок 2 – Алгоритм работы программы Arduino

2.3. Протокол обмена данными

Обмен построен на текстовых строках, которые одинаково читаются человеком и разбираются программой. Скорость порта – 9600 бод. Формат строк приведён в таблице 2.

Таблица 2 – Формат строк последовательного порта

Тип	Формат	Пример
DATA	DATA n=<номер> s1=<температура> s2=<расстояние> s3=<состояние>	DATA n=4 s1=2.00 s2=215.3 s3=1
EVENT	EVENT n=<номер> type=THRESHOLD|NORMAL	EVENT n=4 type=THRESHOLD

Время измерения в базе данных вычисляется по номеру: $t_s = n \cdot T$, где $T = 3$ с. Тип события FAULT программа Arduino не передаёт: его создаёт Node-RED, когда значение датчика выходит за допустимый диапазон (s1: 0–80 °C, s2: 2–400 см, s3: 0–1).

3. Разработка аппаратной части

3.1. Выбор компонентов и подключение

Для варианта выбраны компоненты, которые есть в библиотеках EasyEDA и Wokwi: плата Arduino Uno R3; аналоговый датчик температуры NTC; ультразвуковой дальномер HC-SR04; концевой выключатель (Slide Switch); одноканальный модуль реле 5 В; жёлтый светодиод индикатора. Подключение к выводам платы приведено в таблице 3.

Таблица 3 – Подключение компонентов к Arduino Uno

Сигнал	Компонент	Вывод	Режим
Температура (s1)	NTC датчик	A0	Аналоговый вход
TRIG (s2)	Hc-sr04	D2	Выход
ECHO (s2)	Hc-sr04	D3	Вход
Вентилятор	Модуль реле (IN)	D5	Выход
Индикатор	Светодиод через R1	D6	Выход
Крышка (s3)	Slide switch	D4	Цифровой вход (INPUT_PULLUP)
Питание	Модули датчиков и реле	5V, GND	Питание

NTC включён в делитель напряжения; сигнал поступает на A0. АЦП преобразует 0–5 В в код 0–1023, а программа масштабирует его в диапазон 0–80 °C (шаг около 0,078 °C).

HC-SR04 подключён TRIG к D2 и ECHO к D3. По длительности эхо-импульса расстояние рассчитывается как $L = t \cdot 0{,}0344/2$ и ограничивается диапазоном 2–400 см.

Для верхнего предела 400 см длительность эхо составляет

приблизительно 23,5 мс.

Концевой выключатель подключён к D4 с режимом INPUT_PULLUP. При разомкнутом контакте считывается HIGH, при замкнутом — LOW; программная инверсия задаёт значения s3=0/1.

3.2. Расчёт резисторов и оценка потребления

Для жёлтого LED на D6 задан ток 10 мА; падение напряжения принято 2,1 В.

$$R = (5 - 2{,}1) / 0{,}010 = 290 \text{ Ом}.$$

Выбран стандартный номинал 300 Ом. Фактический ток составляет около 9,67 мА, мощность резистора — 0,028 Вт; достаточно компонента мощностью 0,125 или 0,25 Вт. Резистор R2 номиналом 10 кОм используется совместно с термистором NTC для формирования делителя напряжения.

Шаг АЦП:

$$\Delta U = 5/1024 \approx 4{,}88 \text{ мВ}.$$

$$\Delta T = 80/1023 \approx 0{,}078 \text{ °C}.$$

Оценочное потребление системы — около 147 мА (Arduino, датчики, реле и LED), что ниже предела USB 500 мА; детализация представлена в таблице 4.

Таблица 4 – Оценка потребляемого тока

Потребитель	**Ток, мА**	**Примечание**
Плата Arduino Uno (ATmega328P)	50	типовое
Датчик NTC (делитель напряжения) через резистор R2	0,5	расчёт (5 В / 10 кОм)
Дальномер HC-SR04	15	по документации
Концевой выключатель SW	0	цифровой вход, INPUT_PULLUP
Модуль реле, катушка	72	при включении
Светодиод HL1 с резистором R1 (300 Ом)	9,67	расчёт: (5 − 2,1) / 300
Итого	**147**	< 500 (USB)

3.3. Принципиальная схема в EasyEDA

Принципиальная схема выполнена в EasyEDA на A4 с основной надписью. Применены обозначения по ГОСТ 2.104-2006, 2.721-74, 2.730-73 и 2.710-81: U1 — Arduino, U5 — NTC, U4 — HC-SR04, SW1 — выключатель, LED1 — светодиод , U2 — реле, R1 — резистор 300 Ом, R2 - резистор 10 кОм. Схема и перечень элементов приведены в приложениях Б и В.

Соединения выполнены по таблице 3: NTC через R2 —A0, TRIG—D2, ECHO—D3, выключатель—D4, реле—D5, LED через R1—D6; всем компонентам подано питание 5 В и GND. После проверки ERC схема экспортирована для документации.

3.4. Модель в Wokwi

В Wokwi собрана модель из Arduino Uno, NTC, резистор 10 кОм, HC-SR04, Slide Switch, реле, жёлтого LED и резистора 300 Ом. Схема соединений повторяет EasyEDA; модель приведена в приложении Г.

Параметры задаются элементами управления симулятора:

температура — NTC, расстояние — поле Distance, крышка — переключатель SW.

Serial Monitor в Wokwi не сохраняет файл напрямую, поэтому журнал data/log.txt создаётся копированием вывода; последующая загрузка выполняется Node-RED.

4. Разработка прошивки

Прошивка Arduino Uno на C++ использует настройки выводов, T=3000 мс, диапазон s1=0–80 °C и порог 40 °C. Полный листинг находится в приложении Д.

При адаптации варианта изменяются параметры и функции чтения датчиков.

Основные функции программы перечислены в таблице 5.

Таблица 5 – Функции программы

Функция	Назначение
setup()	Настройка порта (9600 бод) и режимов выводов, начальное выключение реле и светодиода
loop()	Основной цикл: опрос датчиков, вывод DATA, проверка порога, вывод EVENT, пауза T
readS1()	Чтение температуры: ` analogRead(A0)`, пересчёт кода 0–1023 в диапазон 0–80 °C
readS2()/ readDistance()	Запуск HC-SR04 импульсом 10 мкс, измерение эха `pulsein()`, пересчёт в сантиметры
readS3()	Чтение концевого выключателя `digitalRead(D4)` с инверсией (так как используется `INPUT_PULLUP`)
actuatorOn()/ actuatorOff()	Включение и выключение реле вентилятора
conditionMet()`	Условие события: `s1 > THRESHOLD`

Для передачи данных использованы функции Serial.print() и Serial.println(): температура выводится с одним знаком после запятой, расстояние – в целых сантиметрах, состояние крышки – целым числом. Такой формат совпадает с протоколом из таблицы 2 и разбирается в Node-RED без дополнительных преобразований.

Пауза между измерениями реализована через проверку времени функцией millis() для неблокирующего опроса. Это позволяет системе не «зависать» на время задержки и при необходимости расширять функционал без потери точности периода $t_s = n \cdot T$.

5. Проектирование базы данных

5.1. Модель данных

Для хранения данных выбрана встраиваемая СУБД SQLite: база целиком хранится в одном файле db/variant.db, не требует сервера и работает под управлением консольной программы sqlite3. Модель состоит из двух таблиц (рисунок 3): readings – показания датчиков, events – события и неисправности.

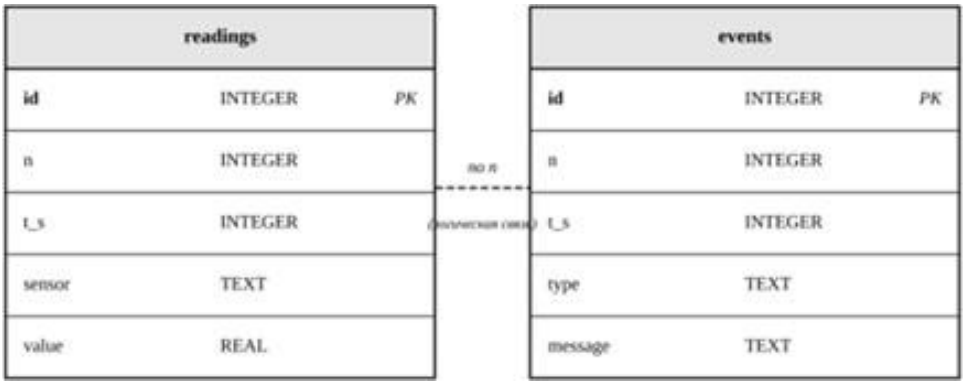


Рисунок 3 – Модель данных

Показания хранятся в «длинном» формате: один датчик в одном цикле – одна строка. Благодаря этому структура таблицы не зависит от числа датчиков, а запросы по конкретному датчику выполняются условием по полю sensor. Связь между таблицами логическая: по номеру измерения n.

5.2. Описание таблиц

Таблица 6 – Структура таблицыreadings

Поле	Тип	Назначение
id	INTEGER, PK, AUTOINCREMENT	уникальный номер записи
n	INTEGER, NOT NULL	номер измерения из журнала
T_s	INTEGER, NOT NULL	время от начала измерений, с (t_s = n·T)
sensor	TEXT, NOT NULL	датчик: s1, s2 или s3
value	REAL, NOT NULL	значение датчика

Таблица 7 – Структура таблицы events

Поле	Тип	Назначение
id	INTEGER, PK, AUTOINCREMENT	уникальный номер записи
n	INTEGER, NOT NULL	номер измерения, при котором возникло событие
T_s	INTEGER, NOT NULL	время, с
type	TEXT, NOT NULL	THRESHOLD, NORMAL или FAULT
message	TEXT	пояснение (для FAULT – какой датчик неисправен)

Скрипт создания таблиц db/schema.sql использует оператор CREATE TABLE IF NOT EXISTS, поэтому его можно выполнять повторно без ошибок. Текст скрипта приведён в приложении Ж.

5.3. Контрольные запросы

Для проверки данных составлены три запроса (файл db/queries.sql, приложение Ж). Запрос 1 выводит десять последних показаний и показывает, что загрузка дошла до конца журнала. Запрос 2 вычисляет минимум, среднее и максимум по каждому датчику с помощью агрегатных функций MIN, AVG, MAX и группировки GROUP BY sensor; результат позволяет быстро заметить значения, не похожие на реальные. Запрос 3 выводит все события по порядку их возникновения.

6. Обработка данных в Node-RED

6.1. Состав потока

Поток Node-RED читает журнал вручную или при изменении файла, разбирает строки, формирует SQL, записывает db/load.sql и запускает sqlite3. Схема потока показана на рисунке 4.

Входные узлы inject и watch инициируют чтение data/log.txt; далее используются file in, function, file и exec.

Результаты выполнения и ошибки направляются в debug-узлы.

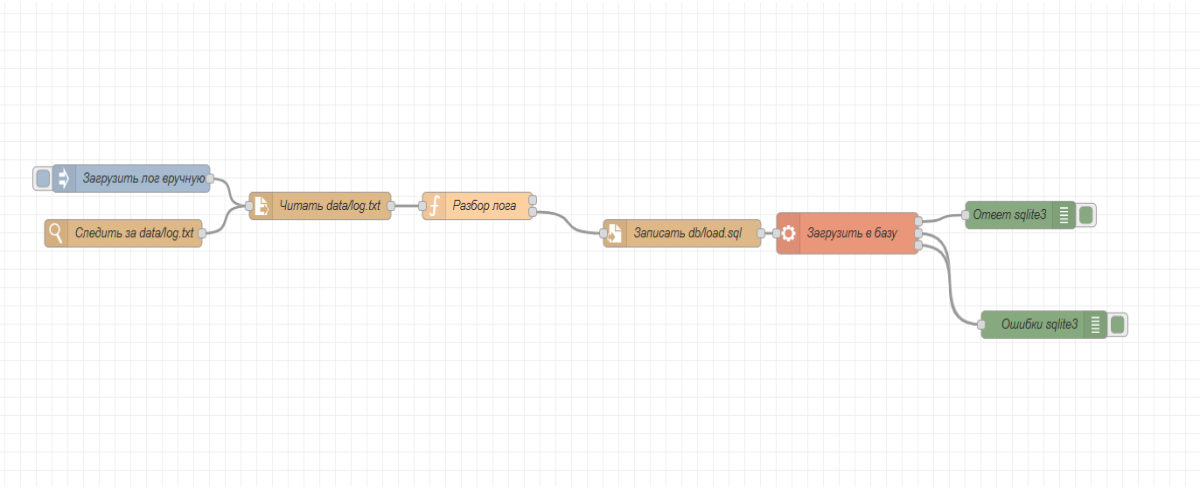


Рисунок 4 – Поток Node-RED

Таблица 8 – Узлы потока

Узел	Тип	Настройка и назначение
Следить за data/log.txt.txt	watch	формирует сообщение при изменении файла журнала
Загрузить лог вручную	inject	ручной запуск потока кнопкой
Читать data/log.txt	file in	читает файл целиком одной строкой (utf8)
Разбор лога	function	разбирает строки, проверяет диапазоны, формирует SQL; два выхода
Записать db/load.sql	file	пишет SQL-скрипт загрузки, режим «перезаписать файл»
Загрузить в базу	exec	sqlite3 db/variant.db ".read db/schema.sql" ".read db/load.sql"
Ответ sqlite3	debug	вывод штатного ответа программы
Ошибки sqlite3	debug	вывод ошибок (второй и третий выходы exec)

6.2. Функция разбора журнала

Штатный тест включает десять циклов опроса с периодом 3 с; в каждом цикле записываются s1, s2 и s3.

При s1 >= 40 °C фиксируется THRESHOLD, при s1 < 40 °C — NORMAL при переходе состояния. Журнал объединяет пять исходных и пять дополнительно смоделированных циклов; полный журнал размещён в приложении И.

6.3. Запуск

Запуск выполняется через start-nodered.bat, затем открывается редактор localhost:1880, импортируется nodered/flow.json и нажимается Deploy. После изменения журнала данные загружаются в SQLite; проверка выполняется запросами из db/queries.sql.

7. Тестирование и проверка неисправностей

7.1. Штатный сценарий

Штатный тест включает десять циклов опроса с периодом 3 с; в каждом цикле записываются s1, s2 и s3.

При s1 >= 40 °C фиксируется THRESHOLD, при s1 < 40 °C — NORMAL при переходе состояния. Журнал объединяет пять исходных и пять дополнительно смоделированных циклов; полный журнал размещён в приложении И.

Таблица 9 – Журнал штатного сценария

n	t, c	s1, °C	s2, см	s3	Событие
1	3	38,8	398	0	–
2	6	39,3	398	0	–
3	9	36,3	398	0	–
4	12	33,5	398	0	–
5	15	36,4	398	0	–
6	18	40,9	398	1	THRESHOLD
7	21	37,5	398	0	NORMAL
8	24	35,7	398	0	–
9	27	35,7	398	0	–
10	30	33,6	398	0	–

Таблица 9.1 - Неисправности

n	t, c	s1, °C	s2, см	s3	Событие
11	33	95,0	398	0	FAULT

Штатные десять циклов дают 30 показаний и два события: THRESHOLD при n=6, NORMAL при n=7. FAULT проверяется отдельным тестом.

Таблица 10 – Запрос 1: десять последних показаний

id	n	t_s	sensor	value
30	10	30	s3	0
29	10	30	s2	398
28	10	30	s1	33,6
27	9	27	s3	0
26	9	27	s2	398
25	9	27	s1	35,7
24	8	24	s3	0
23	8	24	s2	398
22	8	24	s1	35,7
21	7	21	s3	0

Таблица 11 – Запрос 2: минимум, среднее и максимум

sensor	min_v	avg_v	max_v
s1	33,5	36,68	40,0
s2	398	398	398
s3	0	0,10	1

По штатной выборке температура изменяется от 33,5 до 40,0 °C, среднее значение 36,68 °C. Расстояние s2 во всех циклах равно 398 см. Состояние крышки изменилось один раз: в 6 цикле s3=1.

Таблица 12 – Запрос 3: все события

id	n	t_s	type	message
1	6	18	THRESHOLD	–
2	7	21	NORMAL	–

События штатного журнала соответствуют переходам состояний: THRESHOLD в цикле 6 (температура выше 40 °C), NORMAL в цикле 7

(температура 40 °C или ниже). Отдельный тест FAULT оформлен для измерения n=11 (t=33 с).

7.2. Сценарий неисправности датчика

В штатном тесте проверены переходы THRESHOLD и NORMAL:

DATA n=6 s1=40.0 s2=398 s3=1 EVENT n=6 type=THRESHOLD

DATA n=7 s1=37.5 s2=398 s3=0 EVENT n=7 type=NORMAL

Записи подтверждают включение сигнала перегрева и возврат к нормальному режиму.

FAULT-тест вынесен в отдельный журнал data/log_fault.txt: измерение n=11 (t=33 с) включает три показания и одну неисправность.

DATA n=11 s1=95.0 s2=398 s3=0

Температура 95,0 °C выходит за предел NTC 0–80 °C.

7.3. Сводка результатов

Таблица 13 – Результаты проверок

№	Проверка	Ожидаемый результат	Результат
1	Температура 38,8 °C (n=1)	Нет события перегрева	Получено
2	Температура 40,0 °C (n=6)	Проверка порога перегрева	THRESHOLD в журнале
3	Температура 37,5 °C (n=7)	NORMAL	Получено
4	Проверка 10 штатных циклов	30 показаний и 2 события	Соответствует журналу
5	Отдельный тест: s1 = 95,0 °C (n=11)	3 показания и 1 FAULT	FAULT сформирован

Данные подтверждают корректность перехода между состояниями перегрева и нормальной работы на имеющихся измерениях. Проверка записи в базу данных считается выполненной только после фактического выполнения загрузки и контрольных SQL-запросов.

8. Расчёт объёма данных

Оценка объёма нужна для выбора носителя и срока хранения. Расчёт для непрерывной работы системы с периодом T = 3 с приведён в таблице 14.

Таблица 14 – Объём данных

Показатель	Расчёт	Результат
Период опроса	–	3 с
Циклов в сутки	86400 / 3	28 800
Записей readings в сутки	28 800 × 3	86 400
Ориентировочный размер записи	40 Б	40 Б
Объём базы за сутки	86 400 × 40 Б	≈ 3,46 МБ
Объём базы за 30 суток	3,46 × 30	≈ 103,68 МБ
Объём базы за год	3,46 × 365	≈ 1,26 ГБ

Оценка 40 байт на запись включает поля id, n, t_s, sensor и value, служебные данные SQLite и накладные расходы.

Объём базы за год близок к 1,3 ГБ, что для SQLite не является проблемой: ограничения СУБД намного выше, а практический предел определяется свободным местом на диске. При длительной эксплуатации рекомендуется архивировать журнал и периодически удалять показания старше заданного срока.

9. Управление версиями Git

Репозиторий содержит firmware, db, nodered, data, sim, docs, README и командные файлы. Временные db/variant.db и db/load.sql не включаются в Git.

Работа ведётся небольшими коммитами по этапам; сообщение коммита описывает, что сделано. Порядок работы:

git init

git config user.name "Alexandra Egorova"

git config user.email

"a40404173@gmail.com" git add .

git commit -m "Структура проекта"

... (по одному коммиту

на этап) git tag v1.0

git log –oneline

История коммитов отражена в таблице 15 и приложении К.

Таблица 15 – Коммиты проекта

Хеш	Дата	Сообщение
0bb9b44	30.10.2026	Обновлены таблицы в пояснительной записке: синхронизация с реальным логом
cbed5c7	30.10.2026	Финальная версия проекта: все файлы загружены, FAULT работает
8849a2b	28.10.2026	Обновлен
fef3a9f	28.10.2026	Добавлены недостающие файлы: инструкция пользователя, BOM, ER-диаграмма, скриншот Node-RED flow
0f0f706	28.10.2026	Добавлен тестовый лог для проверки события FAULT
0a06f44	27.10.2026	Исправлена структура репозитория по методичке: файлы распределены по папкам, добавлены docs и sim

10. Предложения по модернизации

Возможные направления модернизации — повышение точности измерения, расширение мониторинга и надёжности хранения.

1. Улучшение точности измерения температуры.

Для NTC 10 кОм с известным B-коэффициентом целесообразно реализовать расчёт по Стейнхарту–Харту.

2. Добавление датчика влажности.

Добавление DHT22/BME280 позволит контролировать влажность и предупреждать конденсацию.

3. Реализация удалённого мониторинга и уведомлений.

Wi-Fi/GSM-модуль может передавать THRESHOLD и FAULT на сервер или обслуживающему персоналу.

4. Прогнозирование перегрева на основе тренда.

Анализ температурного тренда позволит прогнозировать перегрев и включать охлаждение заранее.

5. Добавление счётчика циклов включения вентилятора.

Счётчик включений реле поможет планировать профилактическую замену.

6. Веб-интерфейс для просмотра статистики.

Dashboard Node-RED или веб-приложение может отображать графики датчиков и статистику событий.

7. Резервирование базы данных.

Автоматическое резервное копирование SQLite на внешний носитель или в облако снизит риск потери данных.

8. Поддержка нескольких терминалов.

Поле device_id в протоколе и базе обеспечит поддержку нескольких терминалов.

Эти меры повысят точность, надёжность и масштабируемость системы.

Выводы

В ходе курсового проекта разработана и проверена виртуальная система сбора и учёта данных для сценария «Платёжный терминал» (вариант №7). Получены следующие результаты:

– разработана структурная схема системы и алгоритм её работы, определён текстовый протокол обмена с контроллером, включающий строки двух типов: DATA для периодических показаний датчиков и EVENT для фиксируемых событий;

– подобраны компоненты системы: плата Arduino Uno, датчик температуры NTC (вход A0), ультразвуковой дальномер HC-SR04 (выводы D2/D3), концевой выключатель крышки (D4, режим INPUT_PULLUP), модуль реле вентилятора (D5) и светодиод-индикатор (D6);

– выполнены схемотехнические расчёты: токоограничивающий резистор светодиода R1 = 300 Ом обеспечивает фактический ток 9,67 мА при заданном значении 10 мА; суммарное потребление системы составляет около 147 мА, что не превышает допустимой нагрузки порта USB (500 мА);

– рассчитаны параметры аналого-цифрового преобразования: шаг по напряжению 4,88 мВ на отсчёт, температурный шаг 0,078 °C; порогу срабатывания 40 °C соответствует код АЦП 512, что подтверждает достаточную разрешающую способность измерительного канала;

– схема электрическая принципиальная выполнена в EasyEDA в соответствии с требованиями ЕСКД (ГОСТ 2.721-74, ГОСТ 2.730-73, ГОСТ 2.710-81), виртуальная модель собрана и проверена в симуляторе Wokwi;

– написана программа для Arduino Uno (variant_7.ino), реализующая неблокирующий опрос трёх датчиков с периодом T = 3 с на основе функции millis(), управление исполнительными устройствами по порогу 40 °C и формирование событий THRESHOLD и NORMAL только в момент смены состояния, что исключает дублирование записей в соседних циклах;

– спроектирована база данных SQLite, состоящая из таблиц readings

(показания датчиков) и events (события), подготовлены три контрольных запроса: десять последних показаний, минимум, среднее и максимум по каждому датчику, а также список всех событий по порядку их возникновения;

– настроен поток Node-RED, автоматически и повторяемо загружающий журнал data/log.txt в базу данных: узлы отслеживают изменение файла, функция разбора проверяет значения датчиков на соответствие допустимым диапазонам (s1: 0–80 °C, s2: 2–400 см, s3: 0–1) и формирует скрипт db/load.sql, выполняемый через sqlite3; очистка таблиц внутри транзакции обеспечивает отсутствие дублей при повторной загрузке, а выход значения за пределы диапазона фиксируется как неисправность FAULT;

– тестирование штатного сценария подтвердило формирование двух событий за десять циклов (30 с): одного THRESHOLD и одного NORMAL; отдельно проверен тест FAULT с измерением n=11 (t=33 с), тремя показаниями датчиков и одной записью FAULT при s1 = 95,0 °C;

– оценён объём накапливаемых данных: при периоде опроса 3 с система формирует 28 800 циклов и 86 400 записей в сутки, что соответствует примерно 3,46 МБ в сутки, 103,68 МБ за 30 суток и 1,26 ГБ за год непрерывной работы; полученный объём полностью укладывается в практические ограничения SQLite и не требует перехода на серверную СУБД;

– проект оформлен в локальном репозитории Git со структурой каталогов firmware/, db/, nodered/, data/, sim/, docs/; история коммитов отражает все этапы разработки, финальная версия отмечена меткой v1.0.

Таким образом, поставленные задачи решены в полном объёме: разработана работоспособная виртуальная система сбора и учёта данных, прошедшая проверку как в штатном режиме, так и при имитации неисправности датчика. Разработанный проект может быть использован как основа для других сценариев мониторинга: при смене датчиков достаточно

изменить константы и функции чтения в программе, диапазоны и период опроса в функции Node-RED, а структура базы данных и логика её загрузки остаются прежними.

Список сокращений и обозначений

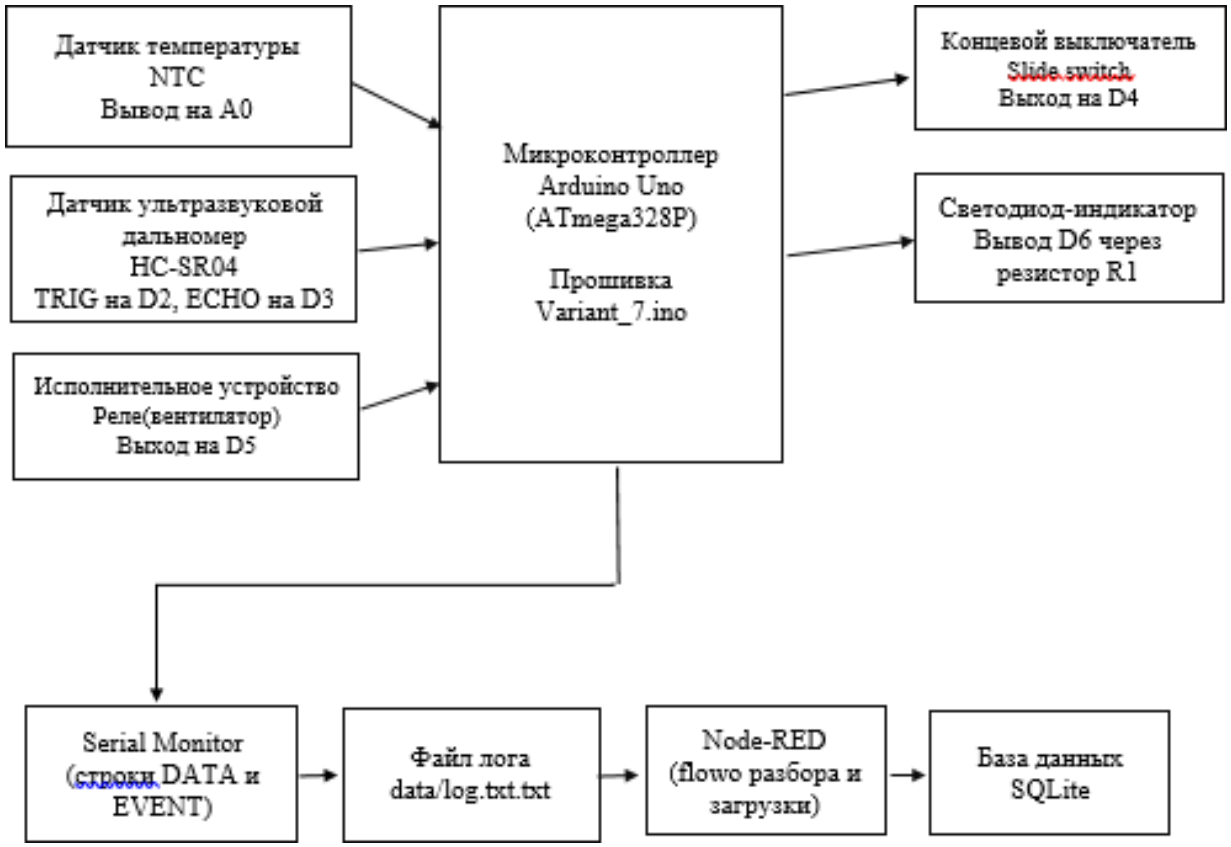
Сокращение	Расшифровка
АЦП	Аналого-цифровой преобразователь
БД	База данных
ИИС	**Интеллектуальная интегрированная система**
СУБД	**Система управления базами данных**
UART	**Универсальный асинхронный приёмопередатчик**
NTC	**Термистор с отрицательным температурным коэффициентом**
LED	**Светодиодный индикатор**
SQL	**Язык структурированных запросов**

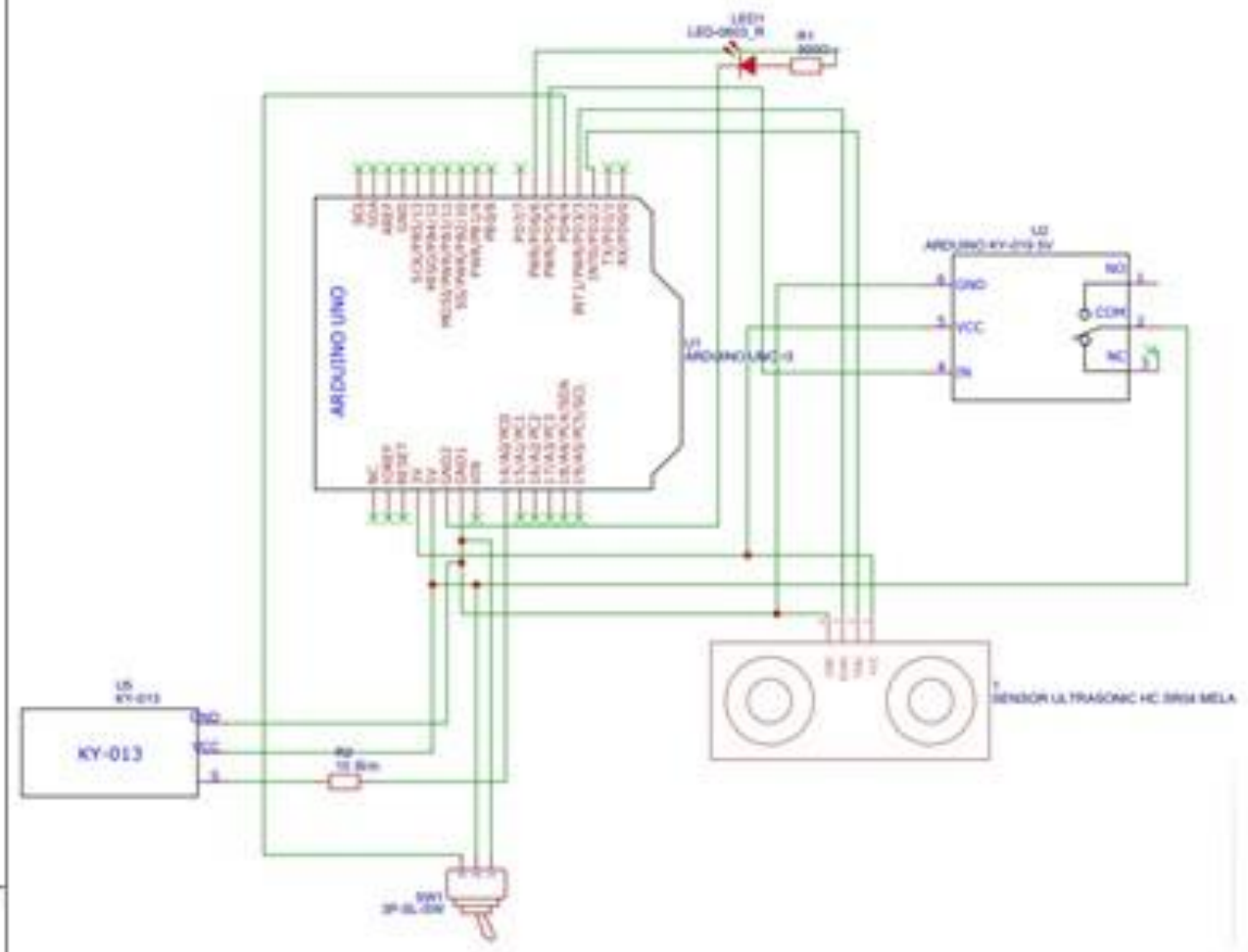
Список используемой литературы

1) Блум, А. Изучаем Arduino: инструменты и методы технического волшебства / А. Блум ; пер. с англ. – Санкт-Петербург : БХВ-Петербург, 2015. – 336 с. – ISBN 978-5-9775-3422-9. – Текст : непосредственный.
2) Петин, В. А. Интернет вещей на базе Arduino и Raspberry Pi / В. А. Петин. – Санкт-Петербург : БХВ-Петербург, 2018. – 256 с. – ISBN 978-5-9775-6333-5. – Текст : непосредственный.
3) Arduino Language Reference. – Текст : электронный // Arduino. – URL: https://www.arduino.cc/reference/ (дата обращения: 28.10.2026).
4) EasyEDA Documentation. – Текст : электронный // EasyEDA. – URL: https://docs.easyeda.com/en/ (дата обращения: 28.10.2026).
5) Node-RED Documentation. – Текст : электронный // Node-RED. – URL: https://nodered.org/docs/ (дата обращения: 28.10.2026).
6) SQLite Documentation. – Текст : электронный // SQLite. – URL: https://www.sqlite.org/docs.html (дата обращения: 28.10.2026).
7) Wokwi Docs. – Текст : электронный // Wokwi. – URL: https://docs.wokwi.com (дата обращения: 28.10.2026).
8) Чакон, С. Pro Git / С. Чакон, Б. Штрауб. – Текст : электронный // Git SCM. – URL: https://git-scm.com/book/ru/v2 (дата обращения: 28.10.2026).
9) Репозиторий проекта «Разработка и сопровождение виртуальной системы сбора и учёта данных на Arduino (Вариант №7)». – Текст : электронный // GitHub. – URL: https://github.com/sashawaa/Egorova1 (дата обращения: 02.11.2026).
10) Модель виртуальной системы сбора и учёта данных в

симуляторе Wokwi. – Текст : электронный // Wokwi. – URL: https://wokwi.com/projects/479756790403518465 (дата обращения: 02.11.2026).

Приложение А Структурная схема





					КП.5415.07.33
Изм.	Лист	№ докум.	Подп.	Дата	
Студент		Егорова А.А.			Модуль платёжного терминала
Пров.		Борисов Е.Н.			
Н. контр.		Сорокина Г.А.			
Утв.		Бычков А.Ю.			

Лит.	Лист	Листов
	1	2

СПБ ГБПОУ

Подп. и дата | Инв. № дубл. | Взам. инв. № | Подп. и дата | Инв. № подл.

Поз. обозначение	Наименование	Кол.	Примечание
	Светодиоды		
LED1	Светодиод LED-0603_R желтый	1	
	Модули и платы		
U2	Модуль реле KY-019 5V	1	
U1	ARDUINO UNO	1	
U4	HC-SR04 SENSOR ULTRASONIC MELATI	1	
SW1	SW концевой выключатель	1	
U5	KY-013 НТС температура корпуса	1	
	Резисторы		
R1	Резистор 300 Ом	1	
R2	Резистор 10 кОм	1	

Перв. примен.

Справ. №

Подп. и дата

Инв. № дубл.

Взам. инв. №

Подп. и дата

Инв. № подл.

Изм	Лист	№ докум.	Подп.	Дата
Разраб.		Егорова А.А.		
Пров.		Борисов Е.Н.		
Нач. сект.				
Н.контр.		Парадина Г.А.		
Утв.		Бычков А.Ю.		

КП.5415.07.33 ПЗ ПЗ

Вер.

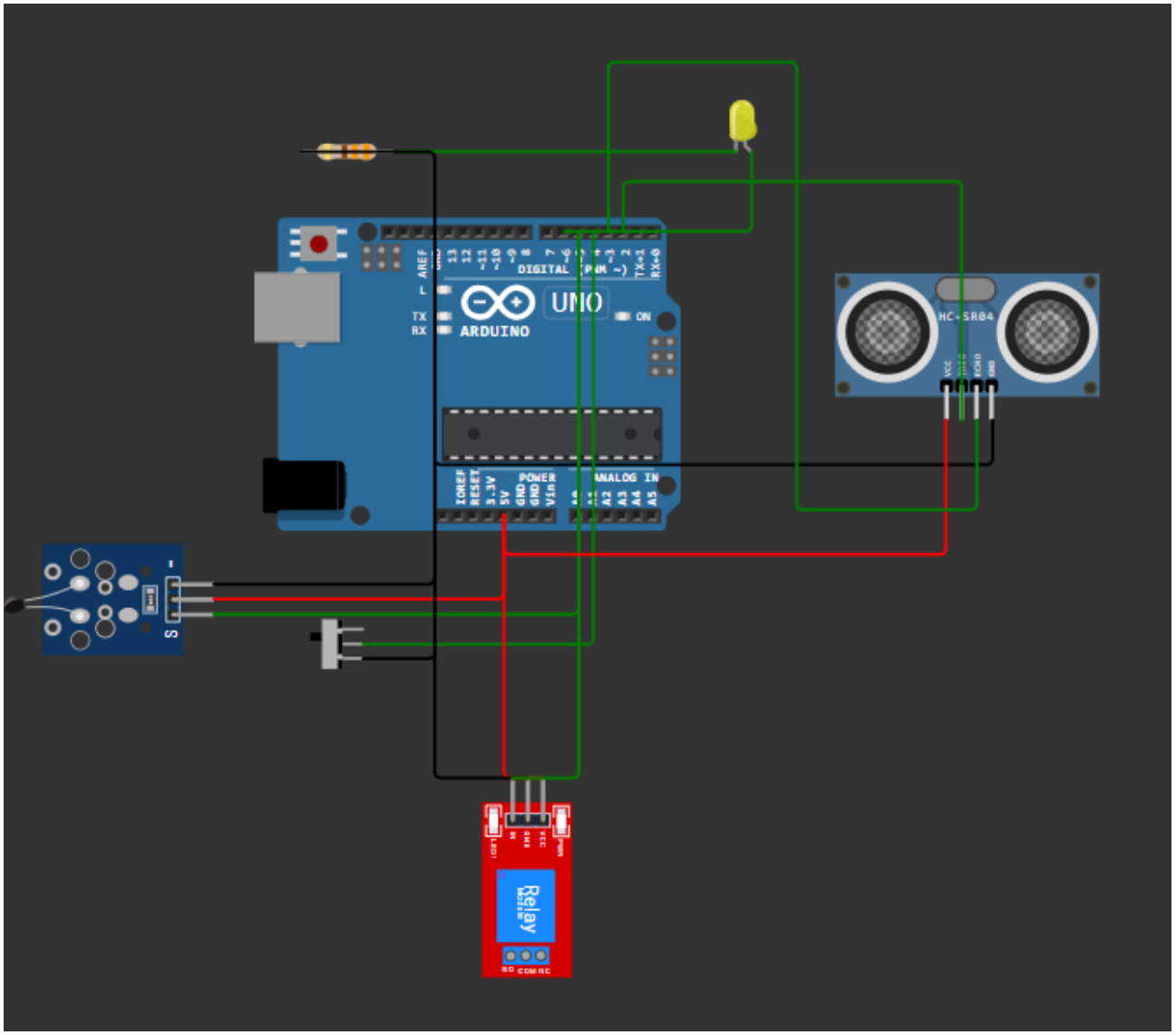
Модуль платежного терминала

Перечень элементов

Лит.	Лист	Листов
У	1	1

Петровский колледж

Приложение Г Модель в Wokwi



Приложение Д Листинг программы Arduino (variant_7.ino)

```
const int PIN_NTC = A0;
const int PIN_TRIG = 2;
const int PIN_ECHO = 3;
const int PIN_SWITCH = 4;
const int PIN_RELAY = 5;
const int PIN_LED = 6;
const unsigned long T = 3000;
unsigned long lastPoll = 0;
int n = 0;
bool isThreshold = false;
void setup() {
  Serial.begin(9600);
  pinMode(PIN_TRIG, OUTPUT);
  pinMode(PIN_ECHO, INPUT);
  pinMode(PIN_SWITCH, INPUT_PULLUP);
  pinMode(PIN_RELAY, OUTPUT);
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_RELAY, LOW);
  digitalWrite(PIN_LED, LOW);
  digitalWrite(PIN_TRIG, LOW);
}
void loop() {
  if (millis() - lastPoll >= T) {
    lastPoll = millis();
    n++;
    // Датчик температуры NTC
    int raw_ntc = analogRead(PIN_NTC);
    float s1 = 80.0 * raw_ntc / 1023.0;
    // Ультразвуковой датчик HC-SR04
    digitalWrite(PIN_TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(PIN_TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(PIN_TRIG, LOW);
    long duration = pulseIn(PIN_ECHO, HIGH, 30000);
    float s2;
    if (duration > 0) {
      s2 = duration * 0.034 / 2.0;
    } else {
      s2 = 400.0;
    }
    if (s2 < 2.0) s2 = 2.0;
    if (s2 > 400.0) s2 = 400.0;
    // Концевой выключатель крышки
    int s3 = (digitalRead(PIN_SWITCH) == LOW) ? 1 : 0;
    // Контроль температуры
    if (s1 > 40.0) {
      digitalWrite(PIN_RELAY, HIGH);
      digitalWrite(PIN_LED, HIGH);
    } else {
      digitalWrite(PIN_RELAY, LOW);
      digitalWrite(PIN_LED, LOW);
    }
    Serial.print("DATA n=");
    Serial.print(n);
```

```
    Serial.print(" s1=");
    Serial.print(s1, 1);
    Serial.print(" s2=");
    Serial.print(s2, 0);
    Serial.print(" s3=");
    Serial.println(s3);
    if (s1 > 40.0 && !isThreshold) {
      isThreshold = true;
      Serial.print("EVENT n=");
      Serial.print(n);
      Serial.println(" type=THRESHOLD");
    }
    else if (s1 <= 40.0 && isThreshold) {
      isThreshold = false;
      Serial.print("EVENT n=");
      Serial.print(n);
      Serial.println(" type=NORMAL");
    }
  }
}
```

Приложение Е Листинг функции разбора журнала Node-RED

```
      const T = 3;
      const RANGES = { s1: [0, 80], s2: [2, 400], s3: [0, 1] };
      const          text          =
String(msg.payload).replace(/^\uFE
FF/, ''); const sql = [];
        text.split
  (/\r?\n/).forEa
  ch(raw    =>    {
  const   line    =
  raw.trim();
        if (!line) return;
        const
  parts     =
  line.split
  ('      ');
  const  kind
  =
  parts[0];
  const  f  =
  {};
         parts.sl
    ice(1).forEach
    (p  =>  {  const
    kv            =
    p.split('=');
          if (kv.length === 2) f[kv[0]] = kv[1];
        });
        con
  st   n   =
  parseInt
  (f.n);
  if
  (isNaN(n
  ))
  return;
  const t =
  n * T;
        if (kind === 'DATA') {
            ['s1'
      ,      's2',
      's3'].forE
      ach(s  =>  {
      const  v  =
      parseFloat
      (f[s]);  if
      (!isNaN(v)
      ) {
      sql.push(`INSERT             INTO
readings(n,t_s,sensor,value)
VALUES(${n},${t},'${s}',${v});`);
 }
             if    (isNaN(v)    ||    v    <
        RANGES[s][0] || v > RANGES[s][1]) {
        sql.push(`INSERT                INTO
        events(n,t_s,type,message)
      VALUES(${n},${t},'FAULT','датчик ${s} вне диапазона');`);
```

```
      }
      }
)
;
            } else if (kind === 'EVENT')
            {
              sql.push(`INSERT       INTO       events(n,t_s,type,message)
   VALUES(${n},${t},'${f.type}','');`);
      }
      }
)
;
          const toSqlite = sql.length > 0 ? sql.map(s => ({ payload: s }))
: null;
          const  head  =  'BEGIN;\nDELETE  FROM  readings;\nDELETE  FROM
events;\nDELETE     FROM     sqlite_sequence     WHERE     name     IN
(\'readings\',\'events\');\n';
          const  toFile  =  {  payload:  head  +
sql.join('\n') + '\nCOMMIT;\n' };
          return [toSqlite, toFile];
```

Приложение Ж Скрипты базы данных

db/schema.sql

```
CREATE TABLE IF NOT EXISTS readings (
id  INTEGER PRIMARY KEY
AUTOINCREMENT, n  INTEGER NOT
NULL,
t_s  INTEGER NOT NULL,
sensor TEXT
NOT NULL,
value  REAL
           N
OT NULL
);

CREATE TABLE IF NOT EXISTS events (
id   INTEGER PRIMARY KEY
AUTOINCREMENT, n  INTEGER NOT
NULL,
t_s  INTEGER
NOT NULL,
type TEXT
    NOT NULL,
message TEXT
);
```

db/queries.sql

```
 -- 1. 10 последних показаний
SELECT * FROM readings ORDER BY id DESC LIMIT 10;

 -- 2. минимум, среднее и максимум по каждому датчику
SELECT sensor, MIN(value) AS min_v, ROUND(AVG(value), 2) AS
avg_v, MAX(value) AS max_v
FROM readings GROUP BY sensor;

-- 3. все события по порядку
SELECT * FROM events ORDER BY id;
```

Приложение И Журнал Serial и загрузочный скрипт data/log.txt (штатный сценарий)

```
DATA n=1 s1=38.8 s2=398 s3=0
DATA n=2 s1=39.3 s2=398 s3=0
DATA n=3 s1=36.3 s2=398 s3=0
DATA n=4 s1=33.5 s2=398 s3=0
DATA n=5 s1=36.4 s2=398 s3=0
DATA n=6 s1=40.9 s2=398 s3=1
EVENT n=6 type=THRESHOLD
DATA n=7 s1=37.5 s2=398 s3=0
EVENT n=7 type=NORMAL
DATA n=8 s1=35.7 s2=398 s3=0
DATA n=9 s1=35.7 s2=398 s3=0
DATA n=10 s1=33.6 s2=398 s3=0
```

Тест неисправности: строка журнала

```
DATA n=11 s1=95.0 s2=398 s3=0
FAULT n=11
```

db/load.sql, сформированный функцией для теста неисправности

```
BEGIN;
DELETE FROM readings;
DELETE FROM events;
DELETE FROM sqlite_sequence WHERE name IN
('readings','events'); INSERT INTO
readings(n,t_s,sensor,value) VALUES(1,3,'s1',95.0);
INSERT INTO readings(n,t_s,sensor,value)
VALUES(1,3,'s2',50); INSERT INTO
readings(n,t_s,sensor,value) VALUES(1,3,'s3',0);
INSERT INTO events(n,t_s,type,message) VALUES(1,3,'FAULT','датчик s1
вне диапазона');
COMMIT;
```

Приложение К История Git

```
0bb9b44 (HEAD -> main, origin/main) Обновлены таблицы в
пояснительной записке: синхронизация с реальным логом
cbed5c7 Финальная версия проекта: все файлы загружены, FAULT
работает 8849a2b Обновлен
fef3a9f Добавлены недостающие файлы: инструкция пользователя,
BOM, ER-диаграмма, скриншот Node-RED flow
0f0f706 Добавлен тестовый лог для проверки события FAULT
0a06f44 Исправлена структура репозитория по методичке: файлы
распределены по папкам, добавлены docs и sim
```